

Kai Yi Ng

Assignment 2

Repo link: <https://github.com/kaiyi03/CWM-Project/tree/main/assignment2>

Q1

```
ubuntu@ubuntu:~/CWM-Project/assignment2$ python3 rdttime.py
CPU time counter: 260166505116602 ns
CPU time diff: 13851 ns
CPU time counter: 260166505116602 ns
CPU time diff: 13851 ns
ubuntu@ubuntu:~/CWM-Project/assignment2$ for i in 1 2 3 4 5 6 7 8 9 10; do python3 rdttime.p
y; done
CPU time counter: 260241719239778 ns
CPU time diff: 16260 ns
CPU time counter: 260241740901820 ns
CPU time diff: 24703 ns
CPU time counter: 260241761785342 ns
CPU time diff: 16408 ns
CPU time counter: 260241781847895 ns
CPU time diff: 21411 ns
CPU time counter: 260241802752580 ns
CPU time diff: 17254 ns
CPU time counter: 260241822796127 ns
CPU time diff: 20208 ns
CPU time counter: 260241844220818 ns
CPU time diff: 15357 ns
CPU time counter: 260241863984270 ns
CPU time diff: 25532 ns
CPU time counter: 260241885255090 ns
CPU time diff: 16594 ns
CPU time counter: 260241904724632 ns
CPU time diff: 20400 ns
```

Minimum: 15357 ns

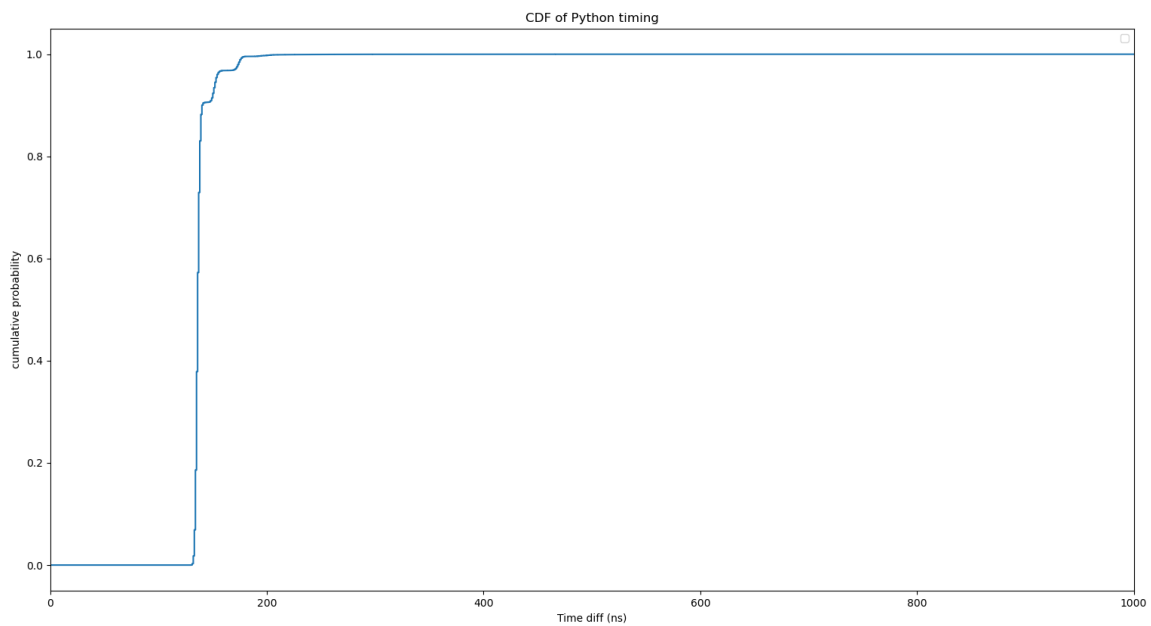
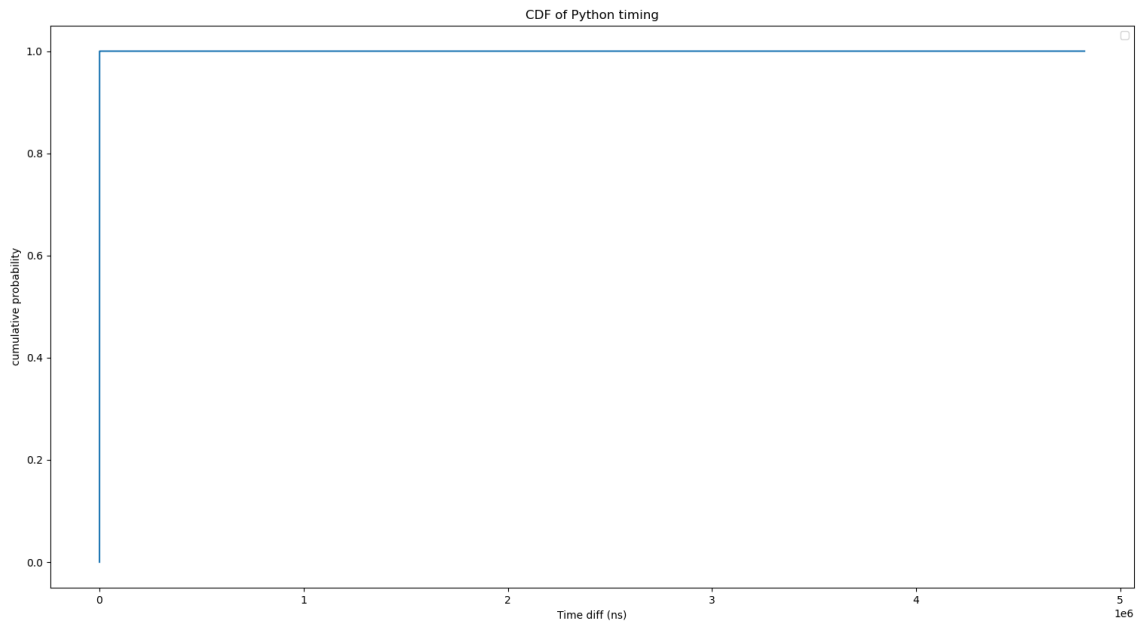
Maximum: 25532 ns

Range: ~10000 ns variation

The difference between the TSC readings basically measures the cost of using the measurement tool.

Q2.

```
ubuntu@ubuntu:~/CWM-Project/assignment2$ python3 rdttime.py
CPU time counter: 260713445548717 ns
CPU time diff: 20057 ns
CPU min diff time: 129 ns
ubuntu@ubuntu:~/CWM-Project/assignment2$
```



Majority of the reading is clustered at 150ns

The flat tail represents the occasional interruptions that may cause slower reads.

Q3

```
ubuntu@ubuntu:~/CWM-Project/assignment2$ gcc -O2 rdttime.c -o rdttime
ubuntu@ubuntu:~/CWM-Project/assignment2$ ./rdttime
CPU frequency: 1.100 GHz
Minimum consecutive TSC diff: 210 cycles
Approximate time: 190.91 ns
ubuntu@ubuntu:~/CWM-Project/assignment2$
```

```
ubuntu@ubuntu:~/CWM-Project/assignment2$ for i in 1 2 3 4 5 6 7 8 9 10; do ./rdttime; done
CPU frequency: 1.100 GHz
Minimum consecutive TSC diff: 210 cycles
Approximate time: 190.90 ns
CPU frequency: 1.100 GHz
Minimum consecutive TSC diff: 208 cycles
Approximate time: 189.09 ns
CPU frequency: 1.100 GHz
Minimum consecutive TSC diff: 206 cycles
Approximate time: 187.27 ns
CPU frequency: 1.306 GHz
Minimum consecutive TSC diff: 66 cycles
Approximate time: 50.52 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 62 cycles
Approximate time: 17.22 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 62 cycles
Approximate time: 17.22 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 62 cycles
Approximate time: 17.22 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 62 cycles
Approximate time: 17.22 ns
```

Balanced mode: CPU frequency varies. TSC diff higher and more variable. CPU not running at full speed

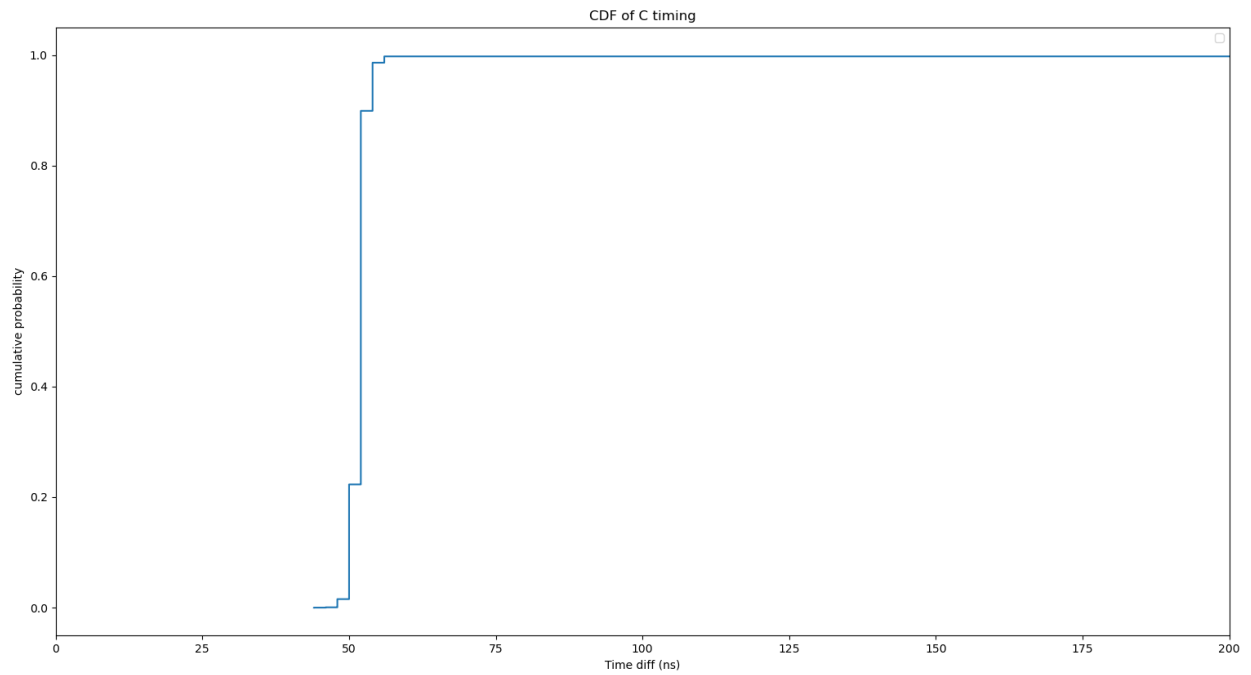
Q4.

```
ubuntu@ubuntu:~/CWM-Project/assignment2$ for i in 1 2 3 4 5 6 7 8 9 10; do ./rdtime; done
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 66 cycles
Approximate time: 18.33 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 66 cycles
Approximate time: 18.33 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 72 cycles
Approximate time: 20.00 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 62 cycles
Approximate time: 17.22 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 52 cycles
Approximate time: 14.44 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 62 cycles
Approximate time: 17.22 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 62 cycles
Approximate time: 17.22 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
ubuntu@ubuntu:~/CWM-Project/assignment2$
```

Performance mode: CPU runs at maximum frequency → TSC diff should be smaller and more consistent → CPU fully powered up

Q5

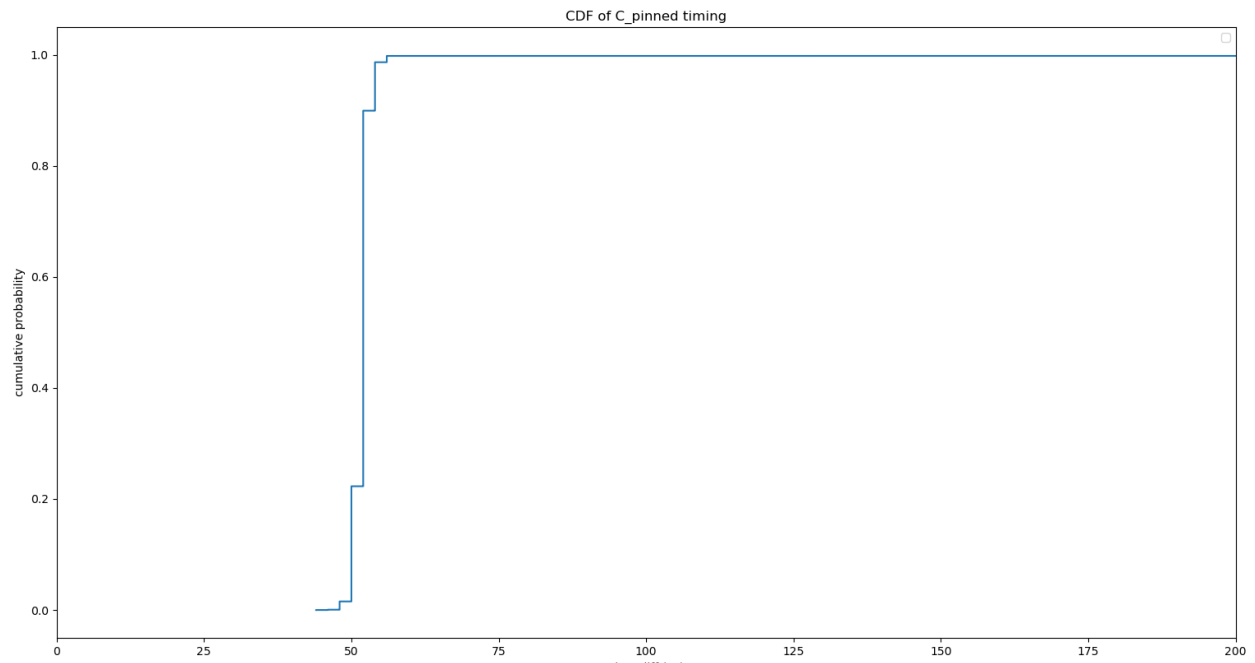
```
ubuntu@ubuntu:~/CWM-Project/assignment2$ cd ~/CWM-Project/assignment2
./rdtime 1000000
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 44 cycles
Approximate time: 12.22 ns
```



Q6

```
ubuntu@ubuntu:~/CWM-Project/assignment2$ taskset -c 1 ./rdtime 1000000
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 44 cycles
Approximate time: 12.22 ns
```

Minimum time diff stays the same as unpinned.



For unpinned:

```
Min: 44
Max: 70812
Mean: 64.1961
Median: 52
P90: 54
P99: 56
```

For pinned:

```
Min: 44
Max: 125404
Mean: 64.9278
Median: 52
P90: 52
P99: 56
```


Maximum number of cycles is higher for pinned. Unpinned program can use other cores when core 1 is busy – better performance.

Q7.

Median: 52 cycles

Mean: 64 cycles

Max: 70812 cycles

The mean is higher than the median because of occasional extreme outliers that pulls the mean up.

Medians may be a better statistic to use for measuring performance.

Maximum is very high because computers have other things in the background that may be fighting for the same processor. Hence, there may be occasional spikes if other programs need to be ran.

Q8.

Python (129ns) is slower than C (12.22ns).

C compiles instructions directly before running. CPU can execute it directly. Hence faster.

Python requires an interpreter to execute each line. Hence extra time

Q9. Significant differences because of other background processes that the computer must run and they use the same CPU and so may occasionally interrupt and cause spikes in times.

Q10. Ping Google DNS

```
ubuntu@ubuntu:~/CWM-Project/assignment2$ ping 8.8.8.8 -c 10 -i 0.2 > ping_google.txt
cat ping_google.txt
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=113 time=3.21 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=113 time=3.28 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=113 time=3.41 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=113 time=3.29 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=113 time=3.27 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=113 time=3.20 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=113 time=3.19 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=113 time=3.33 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=113 time=3.27 ms
64 bytes from 8.8.8.8: icmp_seq=10 ttl=113 time=3.29 ms

--- 8.8.8.8 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 1806ms
rtt min/avg/max/mdev = 3.193/3.273/3.409/0.061 ms
```

Q11.

University of Sydney

```
PING d1puhpggrhw451.cloudfront.net (99.86.12.25) 56(84) bytes of data.
64 bytes from server-99-86-12-25.lhr95.r.cloudfront.net (99.86.12.25): icmp_seq=1 ttl=245 time=4.08 ms
64 bytes from server-99-86-12-25.lhr95.r.cloudfront.net (99.86.12.25): icmp_seq=2 ttl=245 time=4.07 ms
64 bytes from server-99-86-12-25.lhr95.r.cloudfront.net (99.86.12.25): icmp_seq=3 ttl=245 time=4.03 ms
64 bytes from server-99-86-12-25.lhr95.r.cloudfront.net (99.86.12.25): icmp_seq=4 ttl=245 time=4.18 ms
64 bytes from server-99-86-12-25.lhr95.r.cloudfront.net (99.86.12.25): icmp_seq=5 ttl=245 time=4.18 ms
64 bytes from server-99-86-12-25.lhr95.r.cloudfront.net (99.86.12.25): icmp_seq=6 ttl=245 time=4.19 ms
64 bytes from server-99-86-12-25.lhr95.r.cloudfront.net (99.86.12.25): icmp_seq=7 ttl=245 time=4.21 ms
64 bytes from server-99-86-12-25.lhr95.r.cloudfront.net (99.86.12.25): icmp_seq=8 ttl=245 time=4.03 ms
64 bytes from server-99-86-12-25.lhr95.r.cloudfront.net (99.86.12.25): icmp_seq=9 ttl=245 time=4.17 ms
64 bytes from server-99-86-12-25.lhr95.r.cloudfront.net (99.86.12.25): icmp_seq=10 ttl=245 time=3.98 ms

--- d1puhpggrhw451.cloudfront.net ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 1805ms
rtt min/avg/max/mdev = 3.976/4.111/4.210/0.078 ms
```

MIT:

```
PING e9566.dscb.akamaiedge.net (23.43.64.242) 56(84) bytes of data.
64 bytes from a23-43-64-242.deploy.static.akamaitechnologies.com (23.43.64.242): icmp_seq=1 ttl=51 time=3.4
64 bytes from a23-43-64-242.deploy.static.akamaitechnologies.com (23.43.64.242): icmp_seq=2 ttl=51 time=3.4
64 bytes from a23-43-64-242.deploy.static.akamaitechnologies.com (23.43.64.242): icmp_seq=3 ttl=51 time=3.4
64 bytes from a23-43-64-242.deploy.static.akamaitechnologies.com (23.43.64.242): icmp_seq=4 ttl=51 time=3.4
64 bytes from a23-43-64-242.deploy.static.akamaitechnologies.com (23.43.64.242): icmp_seq=5 ttl=51 time=3.5
64 bytes from a23-43-64-242.deploy.static.akamaitechnologies.com (23.43.64.242): icmp_seq=6 ttl=51 time=3.5
64 bytes from a23-43-64-242.deploy.static.akamaitechnologies.com (23.43.64.242): icmp_seq=7 ttl=51 time=3.4
64 bytes from a23-43-64-242.deploy.static.akamaitechnologies.com (23.43.64.242): icmp_seq=8 ttl=51 time=3.4
64 bytes from a23-43-64-242.deploy.static.akamaitechnologies.com (23.43.64.242): icmp_seq=9 ttl=51 time=3.5
64 bytes from a23-43-64-242.deploy.static.akamaitechnologies.com (23.43.64.242): icmp_seq=10 ttl=51 time=3.

--- e9566.dscb.akamaiedge.net ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 1808ms
rtt min/avg/max/mdev = 3.414/3.468/3.556/0.047 ms
```

Oxford:

```
PING www.ox.ac.uk.cdn.cloudflare.net (104.20.34.13) 56(84) bytes of data.  
64 bytes from 104.20.34.13: icmp_seq=1 ttl=52 time=3.73 ms  
64 bytes from 104.20.34.13: icmp_seq=2 ttl=52 time=3.76 ms  
64 bytes from 104.20.34.13: icmp_seq=3 ttl=52 time=4.05 ms  
64 bytes from 104.20.34.13: icmp_seq=4 ttl=52 time=3.92 ms  
64 bytes from 104.20.34.13: icmp_seq=5 ttl=52 time=4.01 ms  
64 bytes from 104.20.34.13: icmp_seq=6 ttl=52 time=4.05 ms  
64 bytes from 104.20.34.13: icmp_seq=7 ttl=52 time=3.94 ms  
64 bytes from 104.20.34.13: icmp_seq=8 ttl=52 time=3.76 ms  
64 bytes from 104.20.34.13: icmp_seq=9 ttl=52 time=3.93 ms  
64 bytes from 104.20.34.13: icmp_seq=10 ttl=52 time=3.83 ms  
  
--- www.ox.ac.uk.cdn.cloudflare.net ping statistics ---  
10 packets transmitted, 10 received, 0% packet loss, time 1806ms  
rtt min/avg/max/mdev = 3.734/3.897/4.052/0.114 ms
```

When I tried to ping University of Moscow it did not receive a response. Likely due to firewalls.

Interesting that the average time for Oxford is longer than MIT even though I am currently at Oxford. Maybe routing is through a hub that is much further.

Q12.

```
ubuntu@ubuntu:~/CWM-Project/assignment2$ ping 127.0.0.1 -c 10 -i 0.2 > ping_localhost.txt
catubuntu@ubuntu:~/CWM-Project/assignment2$ cat ping_localhost.txt
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.021 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.016 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.019 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.018 ms
64 bytes from 127.0.0.1: icmp_seq=5 ttl=64 time=0.018 ms
64 bytes from 127.0.0.1: icmp_seq=6 ttl=64 time=0.021 ms
64 bytes from 127.0.0.1: icmp_seq=7 ttl=64 time=0.019 ms
64 bytes from 127.0.0.1: icmp_seq=8 ttl=64 time=0.020 ms
64 bytes from 127.0.0.1: icmp_seq=9 ttl=64 time=0.018 ms
64 bytes from 127.0.0.1: icmp_seq=10 ttl=64 time=0.017 ms

--- 127.0.0.1 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 1865ms
rtt min/avg/max/mdev = 0.016/0.018/0.021/0.001 ms
```

Small RTT compared to earlier question.

Q13.

```
--- 127.0.0.1 ping statistics ---  
100 packets transmitted, 100 received, 0% packet loss, time 1095ms  
rtt min/avg/max/mdev = 0.007/0.017/0.031/0.003 ms
```

Minimum represents ideal RTT with no interference, very small

Mean is slightly higher than minimum due to occasional interruptions by background computer processes.

Maximum is significantly higher than mean due to occasional interruptions by background processes (jumping between tasks).

Q14.

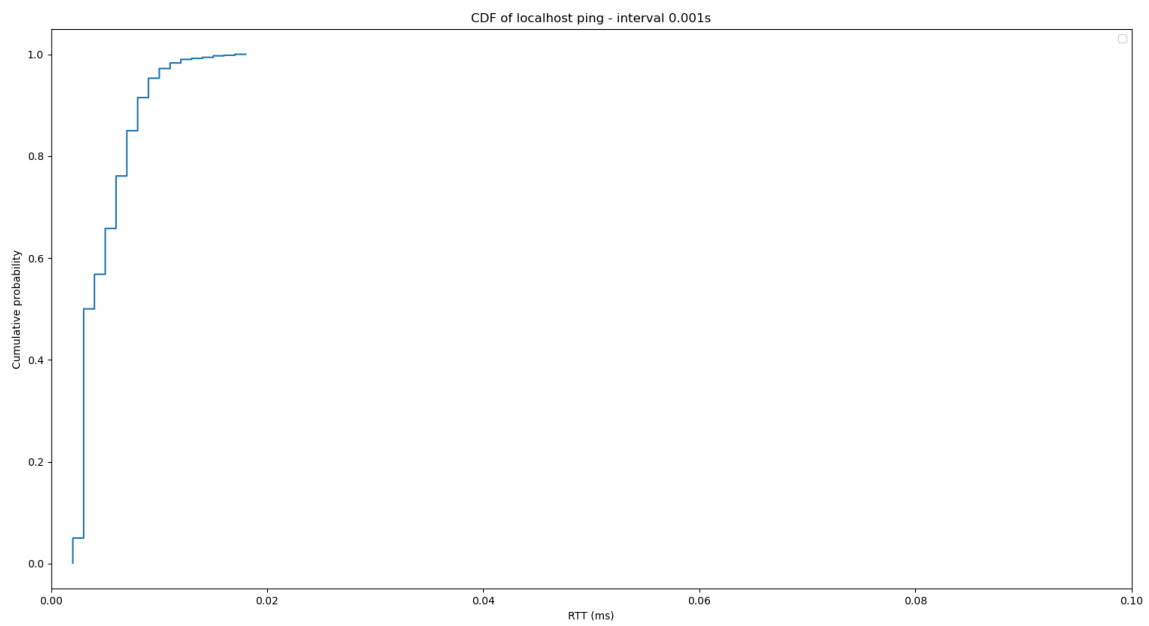
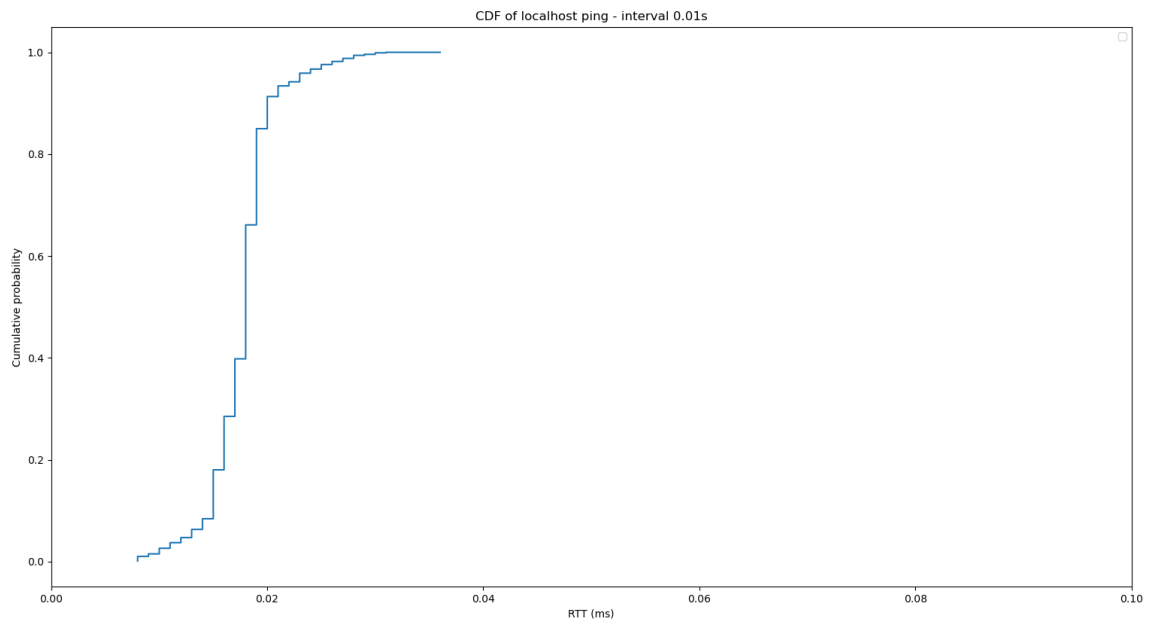
```
ubuntu@ubuntu:~/CWM-Project/assignment2$ sudo ping 127.0.0.1 -c 10000 -f > ping_flood.txt
cat ping_flood.txt
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.

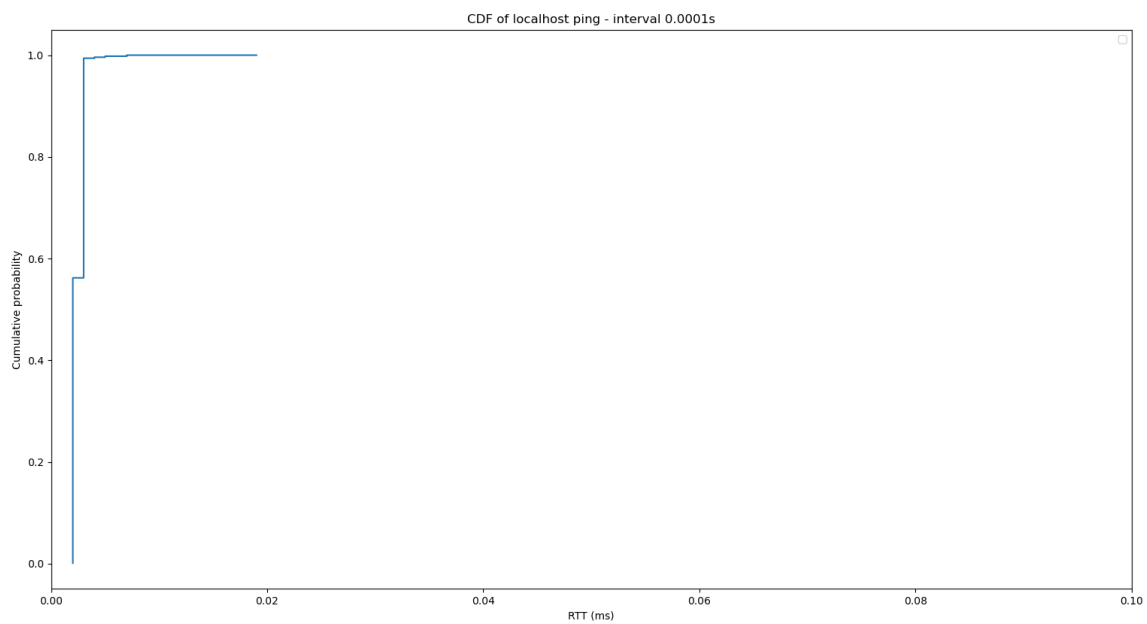
--- 127.0.0.1 ping statistics ---
10000 packets transmitted, 10000 received, 0% packet loss, time 119ms
rtt min/avg/max/mdev = 0.002/0.002/0.313/0.003 ms, ipg/ewma 0.011/0.003 ms
```

Flooding sends packets as fast as possible to test maximum rate of data transmission.

RTT is very small

Q15.





Q16.

Smaller intervals have smaller round-trip results because

At interval 0.0001s, packets are sent so rapidly that the background processes interference observed in earlier questions does not affect, giving low consistent RTT

At interval 0.01s, each packet is processed individually with the CPU performing other tasks between packets. Each packet experiences this overhead cost, giving higher and more variable RTT.

Q17.

Try to change power mode to power saver.

Baseline measurement (performance):

```
ubuntu@ubuntu:~/CWM-Project/assignment2$ ./rdtime 1000000  
CPU frequency: 3.600 GHz  
Minimum consecutive TSC diff: 44 cycles  
Approximate time: 12.22 ns
```

Power saving:

```
ubuntu@ubuntu:~/CWM-Project/assignment2$ ./rdtime 1000000  
CPU frequency: 1.200 GHz  
Minimum consecutive TSC diff: 44 cycles  
Approximate time: 36.67 ns
```

Q18.

Instead of using LFENCE to reduce out of order effects, we can call `__rdtsc()` directly rather than calling `read_tsc`

This reduces the time it takes for LFENCE to run

```
ubuntu@ubuntu:~/CWM-Project/assignment2$ ./rdtime 1000000
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 20 cycles
Approximate time: 5.56 ns
```

Q19.

Number of TSC reads = ping RTT / TSC read time

Average read ping RTT of 3.273ms

Min TSC read time of 17.22ns (Q3)

Number of reads = 190000 (3sf)

Q20.

```
--- Repetition 1 ---  
Single cell time: 12922 ns  
matmul_slow total time: 99478143 ns  
  
--- Repetition 2 ---  
Single cell time: 9029 ns  
matmul_slow total time: 98540738 ns  
n=128 reps=2 checksum=107389.290000
```

matmul_slow() was annotated with time.perf_counter_ns() to measure:

Total function time: ~99ms per repetition

Single cell time: ~9,000-13,000 ns

Each cell requires 128 operations through the Python interpreter.

The first cell is slower (~12,922 ns) than subsequent cells (~9,029 ns) due to cache effects where subsequent cells benefit from data already loaded into cache.

Q21.

```
ubuntu@ubuntu:~/CWM-Project/assignment1$ python3 matmul_fast.py
--- matmul_fast1 ---
matmul_fast1 time: 86484871 ns
matmul_fast1 time: 85479231 ns
--- matmul_fast2 ---
matmul_fast2 time: 88720646 ns
matmul_fast2 time: 88799651 ns
--- matmul_fast3 ---
matmul_fast3 time: 71728731 ns
matmul_fast3 time: 71714541 ns
n=128 reps=2 checksum=0.000000
ubuntu@ubuntu:~/CWM-Project/assignment1$
```

Transposes matrix b first so both matrices are accessed row by row throughout:

row_ai[k]. sequential memory access

row_btj[k]. sequential memory access

Hence fewest cache misses of all three CPU always find data in cache fastest overall despite one time transpose cost

Q22.

I will now be more aware of writing code that accesses memory sequentially rather than randomly to make use of cache.

I can also measure performance before and after changes using tools like time, perf etc. To find out where the bottlenecks are so that i can optimise the correct part. The flame graph showed that the algorithm was the bottle neck but it did not show that python interpreter was actually a significant part that can be optimised.

I can also reduce unnecessary inner loops or removing redundant instructions (e.g. LFENCE)

